

Context Engineering

An answer is only as good as what the model was actually given — missing facts cause it to guess, too much irrelevant text buries what matters.

```
uv run python Day-12-Context-Engineering\example.py
```

Context engineering is deciding, on purpose, what a model sees for one call: the question, the state your app already knows, relevant history, retrieved evidence, and tool results — each one labelled, bounded to a budget, and traceable back to its source.

01 Five context types

TASK

The current user question — it anchors the whole call, and always comes first.

STATE

Execution facts the app already knows — route, status, counters. Only the fields the model needs.

MEMORY

Relevant earlier turns. The model does not remember on its own — the app has to resend this.

RAG

Retrieved, source-labelled passages (like KB-102). Relevance and authorization both matter.

TOOL

The result of an action just taken — current facts, with secrets and noise stripped out first.

Priority when assembling: TASK → TOOL → RAG → MEMORY — current tool output outranks related RAG or memory.

02 Four strategies



Select

Include a RAG passage only if its words overlap the question; include memory only if the request depends on an earlier turn.

02_context_selection.ipynb



Compress

Enforce a hard character budget. Drop the lowest-priority section first — memory, then RAG, then tool. TASK is never dropped.

03_context_compression.ipynb



Offload

Write a large result to external storage. Send only a short reference and summary; read the full text back only if the request needs it.

04_context_offloading.ipynb



Isolate

Give independent specialists their own separate context and their own model call. Merge only the results, never the raw contexts.

05_context_isolation.ipynb